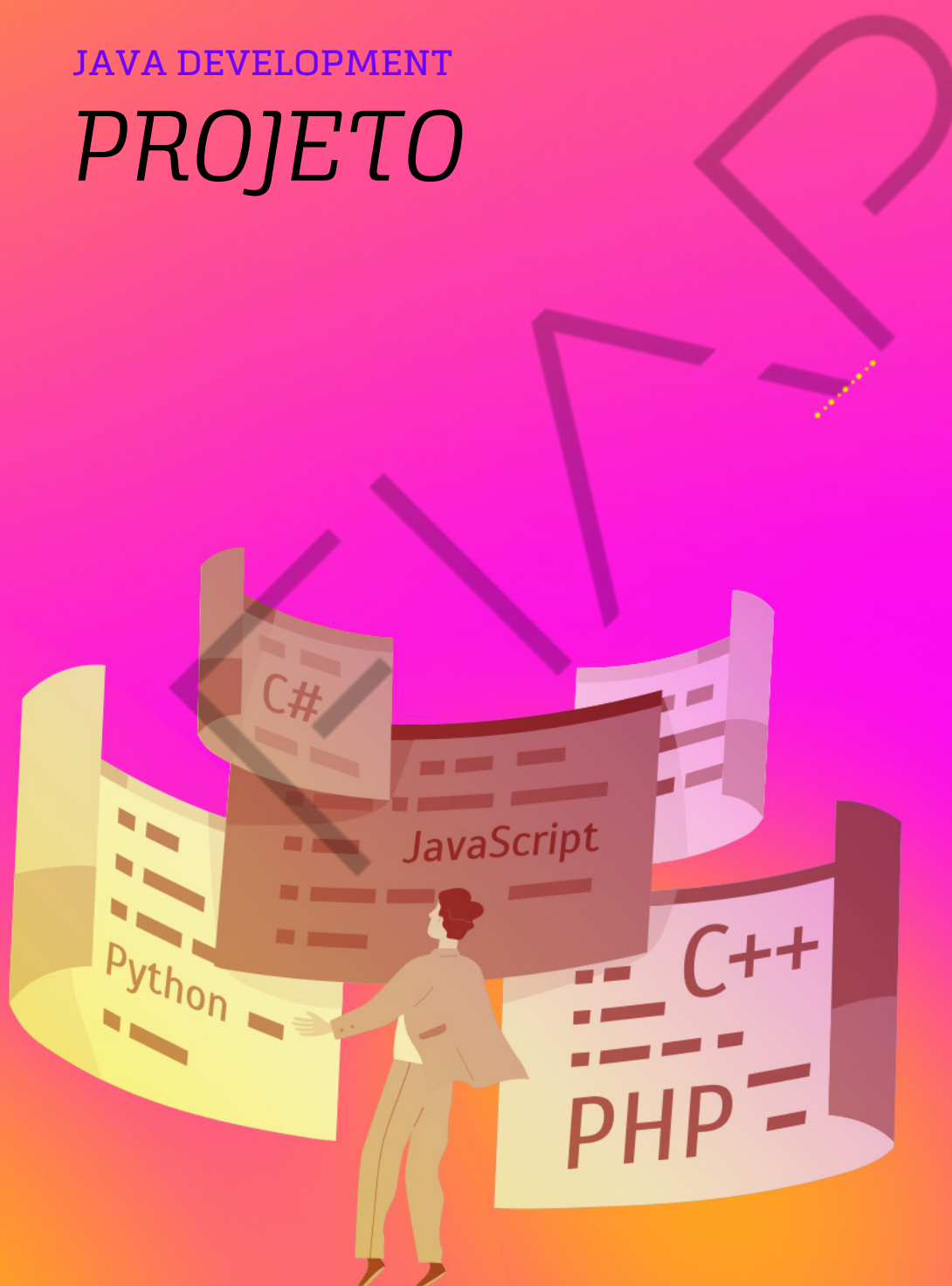




JAVA DEVELOPMENT
PROJETO



6

LISTA DE FIGURAS

Figura 1 - Diagrama de Classes – pacote model	6
Figura 2 – Diagrama de classes - pacote repository	11
Figura 3 - pacote view	19
Figura 4 - O pacote padrão	33
Figura 5 - Menu de opções	39
Figura 6 - Cadastrando uma categoria.....	40
Figura 7 - Sucesso no cadastramento de categoria.....	40
Figura 8 - Categorias existentes	40
Figura 9 - Dando nome ao produto	41
Figura 10 - Inserindo a descrição do produto.....	41
Figura 11 - Inserindo o preço do produto	42
Figura 12 - Mensagem de sucesso ao cadastrar um produto	42
Figura 13 - Alterando dados dos Produtos.....	43
Figura 14 - Tela para alteração da categoria do produto	43
Figura 15 - Alterando o nome do produto	44
Figura 16 - Mensagem de Sucesso	44
Figura 17 - Todos os dados do produto	44
Figura 18 - Consultando produto por Id	45
Figura 19 - Dados do Produto com Id iguala 1.....	45
Figura 20 - Selecionar produto por categoria.....	46
Figura 21 - Selecionando a categoria para listagem.....	46
Figura 22 - Apresentando dados do produto de categoria Celulares.....	47
Figura 23 - Encerrando o sistema	47

LISTA DE CÓDIGOS-FONTE

Código Fonte 1 - A Classe Categoria.....	8
Código Fonte 2 - A classe Produto	10
Código Fonte 3 - A coleção de Categorias	12
Código Fonte 4 - Adicionando o método <i>findAll</i> à coleção de Categorias	12
Código Fonte 5 - Adicionando o método <i>findByld</i>	13
Código Fonte 6 - Método para salvar uma categoria	13
Código Fonte 7 – Versão final da classe <i>Categoria Collection Repository</i>	15
Código Fonte 8 - O repositório de Produtos.....	16
Código Fonte 9 - Adicionando os métodos de Consulta Ao repositório de Produtos	16
Código Fonte 10 - Versão final da classe de repositório de produtos.....	18
Código Fonte 11 - A enumeração de Opções do Sistema.....	20
Código Fonte 12 - A classe Opcao View.....	21
Código Fonte 13 - A classe Categoria View.....	23
Código Fonte 14 - Incluindo o método form à classe Categoria View	23
Código Fonte 15 - Métodos para mensagem de sucesso.....	24
Código Fonte 16 - A versão final da classe Categoria View	24
Código Fonte 17 – Desenvolvendo a classe ProdutoView	25
Código Fonte 18 - O método form para capturar os dados do produto	27
Código Fonte 19 - Método para a atualização de dados do produto	28
Código Fonte 20 - Métodos para mensagem de sucesso.....	29
Código Fonte 21 - Código completo da classe ProdutoView	32
Código Fonte 22 - Iniciando o desenvolvimento da classe App.....	34
Código Fonte 23 - Cadastramento de Categoria	34
Código Fonte 24 - Cadastramento de Produto	35
Código Fonte 25 - Alterar Produto	35
Código Fonte 26 - Consultar produto por Id.....	35
Código Fonte 27 - Consultar produtos por categoria	36
Código Fonte 28 - A classe App.....	37

SUMÁRIO

1	PROJETO.....	5
1.1	O pacote Model.....	6
1.1.1	A classe categoria.....	6
1.1.2	A classe Produto.....	8
1.2	O Pacote Repository.....	11
1.2.1	O repositório de categorias.....	11
1.2.2	O repositório de Produtos.....	15
1.3	O pacote View.....	19
1.3.1	A Enumeração de Opções do Menu.....	19
1.3.2	A classe Opção view.....	21
1.3.3	A classe categoria view.....	22
1.3.4	A classe Produto view.....	25
1.4	O pacote Padrão.....	33
1.4.1	A classe App.....	33
1.5	O programa em funcionamento.....	39
1.5.1	Menu de opções.....	39
1.5.2	Cadastrando uma nova categoria.....	39
1.5.3	Cadastrando um novo produto.....	40
1.5.4	Alterando dados do produto.....	42
1.5.5	Consultando produto por id.....	45
1.5.6	Listando produtos por categoria.....	45
	REFERÊNCIAS.....	48

1 PROJETO

Uma microempresa da área de comércio de equipamentos eletrônicos demandou a necessidade do desenvolvimento de um programa que permita o cadastramento de produtos.

Cada produto deve estar atrelado a uma categoria, mas uma categoria poderá ter diversos produtos. Um produto deverá ter os seguintes atributos:

Nome;

Descrição;

Categoria;

Data de cadastro;

Preço.

1.1 O pacote Model

Para se modelar programas que serão desenvolvidos em linguagem de programação que siga o paradigma da orientação a objetos, utilizamos o Diagrama de Classes da Linguagem de Modelagem Unificada – UML.

O nosso programa terá duas classes que representam as entidades que o sistema deverá controlar. Essas classes de modelo ficarão organizadas no pacote model do nosso projeto.

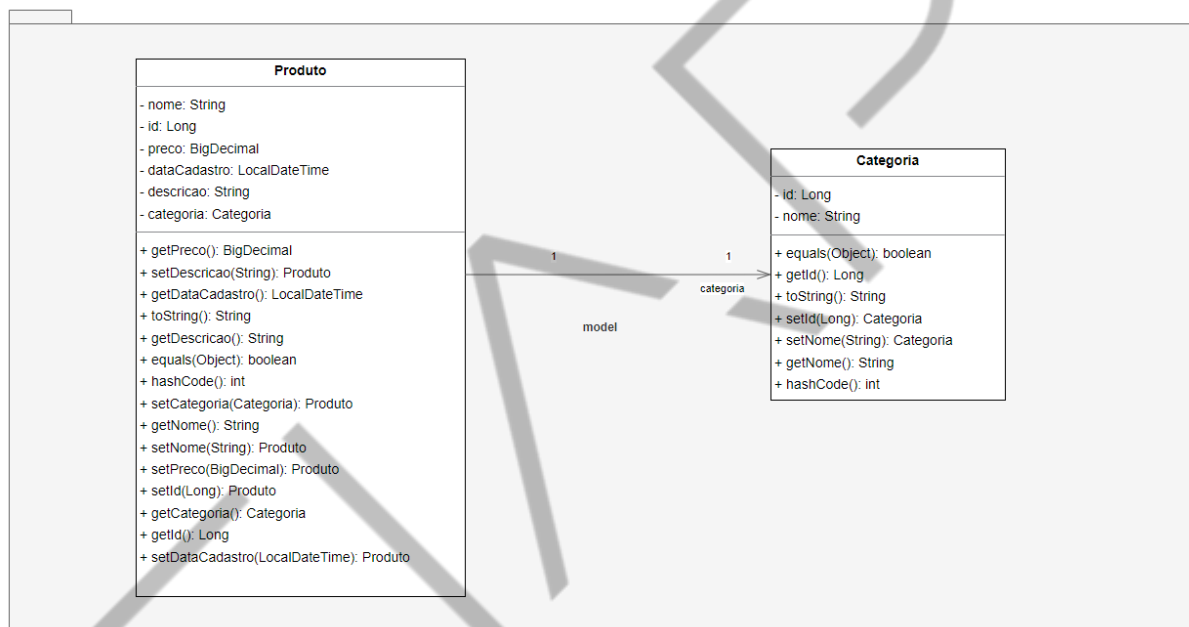


Figura 1 - Diagrama de Classes – pacote model
Fonte: Autor (2023)

Todo produto deverá estar classificado em uma categoria. Dessa maneira, podemos dizer que um produto contém uma categoria. E a relação entre eles é que o produto saiba a qual categoria pertence, mas uma categoria não saberá quais produtos estão ligados a ela ou seja quais são os produtos com a sua categoria. Por esse motivo, na imagem acima, a seta parte de produto e aponta para categoria. Isso demonstra o sentido da informação.

1.1.1 A CLASSE CATEGORIA

A classe categoria nos auxilia a organizar os mais diversos produtos que nosso sistema poderá ter. Contém dois atributos:

Projeto

- Id que serve para identificar a categoria. Cada categoria salva receberá um Id único;
- nome, para guardar o nome da categoria.

Terá três métodos construtores para serem usados conforme a necessidade do utilizador da classe. Terá também métodos assessores, o método *toString*, *equals* e *hashCode*.

```
import java.util.Objects;

public class Categoria {
    private Long id;
    private String nome;

    public Categoria() {
    }

    public Categoria(String nome) {
        this.setNome(nome);
    }

    public Categoria(Long id, String nome) {
        this.setId(id);
        this.setNome(nome);
    }

    public Long getId() {
        return id;
    }

    public Categoria setId(Long id) {
        this.id = id;
        return this;
    }

    public String getNome() {
        return nome;
    }

    public Categoria setNome(String nome) {
        this.nome = nome;
        return this;
    }

    @Override
    public String toString() {
        return getNome().toUpperCase();
    }
}
```

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Categoria categoria = (Categoria) o;
    return Objects.equals(getNome(), categoria.getNome());
}

@Override
public int hashCode() {
    return Objects.hash(getNome());
}
}
```

Código Fonte 1 - A Classe Categoria
Fonte: Autor (2023)

1.1.2 A CLASSE PRODUTO

A classe que representará um produto no nosso programa terá seis atributos básicos para qualquer produto.

```
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.Objects;

public class Produto {
    private Long id;
    private String nome;
    private String descricao;
    private BigDecimal preco;
    private Categoria categoria;
    private LocalDateTime dataCadastro;

    public Produto() {
    }

    public Produto(Long id, String nome, String descricao, BigDecimal preco,
        Categoria categoria, LocalDateTime dataCadastro) {
        this.setId(id);
        this.setNome(nome);
        this.setDescricao(descricao);
        this.setPreco(preco);
        this.setCategoria(categoria);
        this.setDataCadastro(dataCadastro);
    }

    public Produto(String nome, String descricao, BigDecimal preco, Categoria
        categoria, LocalDateTime dataCadastro) {
        this.setNome(nome);
    }
}
```



```
this.setDescricao(descricao);
this.setPreco(preco);
this.setCategoria(categoria);
this.setDataCadastro(dataCadastro);
}

@Override
public String toString() {
    return getNome();
}

public Long getId() {
    return id;
}

public Produto setId(Long id) {
    this.id = id;
    return this;
}

public String getNome() {
    return nome;
}

public Produto setNome(String nome) {
    this.nome = nome;
    return this;
}

public String getDescricao() {
    return descricao;
}

public Produto setDescricao(String descricao) {
    this.descricao = descricao;
    return this;
}

public BigDecimal getPreco() {
    return preco;
}

public Produto setPreco(BigDecimal preco) {
    this.preco = preco;
    return this;
}

public Categoria getCategoria() {
    return categoria;
}

public Produto setCategoria(Categoria categoria) {
    this.categoria = categoria;
    return this;
}
```

```
public LocalDateTime getDataCadastro() {  
    return dataCadastro;  
}  
  
public Produto setDataCadastro(LocalDateTime dataCadastro) {  
    this.dataCadastro = dataCadastro;  
    return this;  
}  
  
@Override  
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (o == null || getClass() != o.getClass()) return false;  
    Produto produto = (Produto) o;  
    return nome.equals(produto.nome) && categoria.equals(produto.categoria);  
}  
  
@Override  
public int hashCode() {  
    return Objects.hash(nome, categoria);  
}  
}
```

Código Fonte 2 - A classe Produto
Fonte: Autor (2023)

Ótimo. Já temos as classes que suprem as necessidades negociais do nosso projeto. Agora precisamos pensar em como os dados serão armazenados. Precisamos pensar em classes que tenham como tarefa executar métodos que manipulam dados independentemente do Sistema Gerenciador de Banco de Dados (SGBD), pois independentemente do SGBD que seja utilizado, um conjunto de classes deve ser capaz de receber e processar solicitações de CRUD, que é o acrônimo para Create (criar), Read (ler), Update (atualizar) e Delete (apagar).

Na verdade, o que precisamos é de um conjunto de classes que funcionem como um repositório.

Sendo assim, seria bem oportuno criar um pacote em nosso projeto para representar esse repositório e organizar as classes com a mesma responsabilidade.

1.2 O Pacote Repository

Criaremos então o pacote “*repository*” e nesse pacote, teremos duas classes. Uma para cada classe de entidade. Dessa forma, cada uma delas terá a responsabilidade de persistir e recuperar dados persistidos da referida classe.

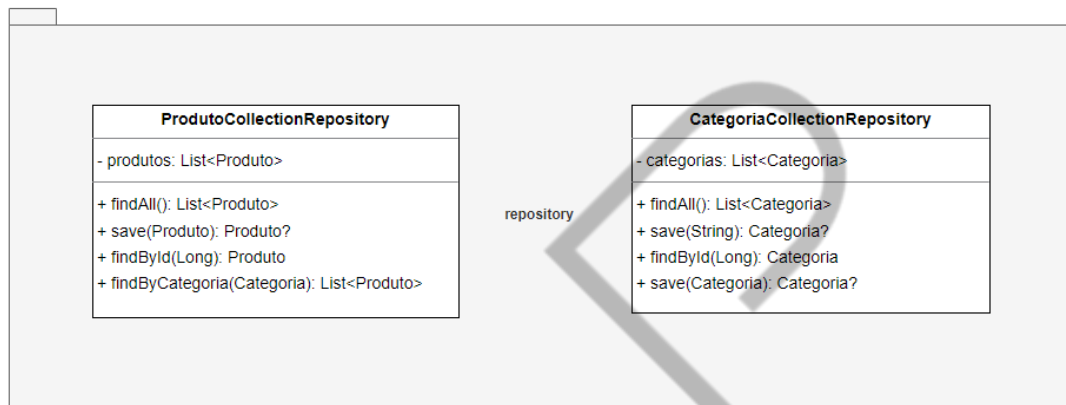


Figura 2 – Diagrama de classes - pacote repository
Fonte: Autor (2023)

Neste projeto, nós não utilizaremos um SGBD, pois conceitos de banco de dados não é o escopo deste curso. Para substituir uma SGBD, trabalharemos com a API de Coleções do Java, pois, com apoio dessa API, nós seremos capazes de simular o comportamento de um repositório. Entretanto, esse repositório será volátil para as categorias que forem criadas durante o funcionamento do programa (excelente para testes em desenvolvimento).

Criaremos, portanto, duas classes: `CategoriaCollectionRepository` e `ProdutoCollectionRepository`.

1.2.1 O repositório de categorias

Iniciaremos agora a construção do repositório de categorias. O código será construído passo a passo aqui. Iniciaremos criando uma Lista de Categorias e depois um bloco estático para que o programa seja iniciado com um conjunto mínimo de categorias (este conjunto mínimo não é volátil). Então vamos aos códigos:

```
import br.com.fiap.nano.produtos.model.Categoria;

import java.util.Arrays;
import java.util.List;
import java.util.Vector;

public class CategoriaCollectionRepository {

    private static List<Categoria> categorias;

    static {
        categorias = new Vector<>();
        String[] categorias = {"Eletrônicos", "Celulares", "Livros", "Games"};
        Arrays.asList(categorias).forEach(CategoriaCollectionRepository::save);
    }

    public static Categoria save(String s) {
        Categoria c = new Categoria(s);
        if (!categorias.contains(c)) {
            c.setId((long) categorias.size() + 1);
            categorias.add(c);
            return c;
        } else {
            return null;
        }
    }
}
```

Código Fonte 3 - A coleção de Categorias
Fonte: Autor (2023)

Já temos uma coleção de Categorias para disponibilizar para o nosso programa. Excelente! O próximo passo é criar um método para consultar todas as categorias:

```
public static List<Categoria> findAll() {
    return categorias;
}
```

Código Fonte 4 - Adicionando o método *findAll* à coleção de Categorias
Fonte: Autor (2023)

Trazer todas as categorias foi muito simples, pois tivemos apenas que retornar a propriedade categorias da nossa listagem. Bom, o passo seguinte é ser capaz de trazer uma categoria a partir do número de Id informado. Vamos lá:

```
public static Categoria findById(Long id) {  
    return categorias.stream()  
        .filter(c -> c.getId().equals(id))  
        .findFirst()  
        .orElse(null);  
}
```

Código Fonte 5 - Adicionando o método findById
Fonte: Autor (2023)

No código `findById`, que tem a missão de buscar uma categoria dentro da lista de categorias, eu transformei momentaneamente a listagem de categorias numa *stream* e fiz uma filtragem nos elementos verificando qual deles possuía o id igual ao id que recebemos como parâmetro do método. Como só pode ter uma categoria com o id, ao encontrar o primeiro, o processamento será encerrado. Caso não exista uma categoria cadastrada com o id recebido como parâmetro o método `orElse` será chamado e irá retornar *null*.

Maravilha, estamos evoluindo rápido! Agora podemos criar o método para salvar uma categoria. Esse método receberá um objeto do tipo categoria e terá uma lógica para determinar o id da nova categoria cadastrada e, em seguida, incluirá a nova categoria na coleção. Vamos aos códigos novamente.

```
public static Categoria save(Categoria categoria) {  
    if (!categorias.contains(categoria)) {  
        categoria.setId((long) categorias.size() + 1);  
        categorias.add(categoria);  
        return categoria;  
    } else {  
        return null;  
    }  
}
```

Código Fonte 6 - Método para salvar uma categoria
Fonte: Autor (2023)

Com isso, concluímos o desenvolvimento da classe com a missão de ser o nosso repositório de categorias.

Veja abaixo a versão final da classe:

Projeto

```
package br.com.fiap.nano.produtos.view;

import br.com.fiap.nano.produtos.model.Categoria;

import java.util.Arrays;
import java.util.List;
import java.util.Vector;

public class CategoriaCollectionRepository {

    private static List<Categoria> categorias;

    static {

        categorias = new Vector<>();
        String[] categorias = {
            "Eletrônicos",
            "Celulares",
            "Livros",
            "Games"};

        Arrays.asList(categorias)
            .forEach(CategoriaCollectionRepository::save);
    }

    public static Categoria save(String s) {
        Categoria c = new Categoria(s);
        if (!categorias.contains(c)) {
            c.setId((long) categorias.size() + 1);
            categorias.add(c);
            return c;
        } else {
            return null;
        }
    }

    public static List<Categoria> findAll() {
        return categorias;
    }

    public static Categoria findById(Long id) {
        return categorias.stream()
            .filter(c -> c.getId().equals(id))
            .findFirst()
            .orElse(null);
    }

    public static Categoria save(Categoria categoria) {
        if (!categorias.contains(categoria)) {
            categoria.setId((long) categorias.size() + 1);
            categorias.add(categoria);
            return categoria;
        } else {
            return null;
        }
    }
}
```

```
}  
}
```

Código Fonte 7 – Versão final da classe *Categoria Collection Repository*
Fonte: Autor (2023)

Se você for atento, perceberá que fizemos uma sobrecarga do método `save`. Pois nessa classe, passamos agora a ter duas maneiras de se salvar uma categoria. A primeira recebe uma *String* como parâmetro, já a segunda maneira recebe um objeto do tipo categoria. O código é bem parecido, inclusive.

1.2.2 O REPOSITÓRIO DE PRODUTOS

Criaremos agora o nosso repositório de produtos. Iniciaremos de forma bem parecida com o que fizemos para o repositório de categorias. Criamos uma lista de produtos, alguns produtos para que o programa tenha uma pequena massa de dados e o método para salvar essa lista padrão de produtos.

Vamos lá:

```
import br.com.fiap.nano.produtos.model.Categoria;  
import br.com.fiap.nano.produtos.model.Produto;  
  
import java.math.BigDecimal;  
import java.time.LocalDateTime;  
import java.util.Arrays;  
import java.util.List;  
import java.util.Vector;  
  
public class ProdutoCollectionRepository {  
  
    private static List<Produto> produtos;  
  
    static {  
        produtos = new Vector<>();  
  
        Produto kindle = new Produto();  
        kindle.setCategoria(CategoriaCollectionRepository.findById(1))  
            .setNome("kindle")  
            .setDescricao("e-reader da Amazon")  
            .setDataCadastro(LocalDateTime.now())  
            .setPreco(BigDecimal.valueOf(899.99));  
  
        Produto iphone = new Produto();  
        iphone.setCategoria(CategoriaCollectionRepository.findById(2))  
            .setNome("Iphone 14 PRO MAX")  
            .setDescricao("Aparelho celular de última geração da Apple")  
            .setDataCadastro(LocalDateTime.now());  
    }  
}
```

```
.setPreco(BigDecimal.valueOf(12999.99));

Arrays.asList(kindle, iphone).forEach(ProdutoCollectionRepository::save);
}

public static Produto save(Produto produto) {
    if (!produtos.contains(produto)) {
        produto.setId((long) produtos.size() + 1);
        produtos.add(produto);
        return produto;
    } else {
        return null;
    }
}
}
```

Código Fonte 8 - O repositório de Produtos
Fonte: Autor (2023)

Repare que adotamos a mesma estratégia. Primeiramente, criamos uma lista de produtos e, em seguida, criamos um bloco estático para termos alguns produtos assim que o nosso sistema seja carregado. Isso é a simulação de alguns produtos salmos num SGBD qualquer. Perceba que criamos dois objetos da classe produto. Um Amazon Kindle e um Apple Iphone 14. Esses produtos estarão disponíveis para consulta assim que o programa for iniciado.

Vamos agora implementar os dois métodos para consulta. Um para consultar os produtos por Id, o segundo, consulta os produtos por categoria e o terceiro, retorna todos os produtos cadastrados.

```
public static Produto findById(Long id) {
    return produtos.stream()
        .filter(p -> p.getId().equals(id))
        .findFirst()
        .orElse(null);
}

public static List<Produto> findByCategoria(Categoria categoria) {
    return produtos.stream()
        .filter(p -> p.getCategoria().equals(categoria))
        .toList();
}

public static List<Produto> findAll() {
    return produtos;
}
```

Código Fonte 9 - Adicionando os métodos de Consulta Ao repositório de Produtos
Fonte: Autor (2023)

Projeto

E com isso concluímos a nossa classe de repositório de produtos. A versão final é exibida abaixo:

```
import br.com.fiap.nano.produtos.model.Categoria;
import br.com.fiap.nano.produtos.model.Produto;
import br.com.fiap.nano.produtos.repository.CategoriaCollectionRepository;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.Arrays;
import java.util.List;
import java.util.Vector;

public class ProdutoCollectionRepository {

    private static List<Produto> produtos;

    static {
        produtos = new Vector<>();

        Produto kindle = new Produto();
        kindle.setCategoria(CategoriaCollectionRepository.findById(1))
            .setNome("kindle")
            .setDescricao("e-reader da Amazon")
            .setDataCadastro(LocalDateTime.now())
            .setPreco(BigDecimal.valueOf(899.99));

        Produto iphone = new Produto();
        iphone.setCategoria(CategoriaCollectionRepository.findById(2))
            .setNome("Iphone 14 PRO MAX")
            .setDescricao("Aparelho celular de última geração da Apple")
            .setDataCadastro(LocalDateTime.now())
            .setPreco(BigDecimal.valueOf(12999.99));

        Arrays.asList(kindle, iphone).forEach(ProdutoCollectionRepository::save);
    }

    public static Produto save(Produto produto) {
        if (!produtos.contains(produto)) {
            produto.setId((long) produtos.size() + 1);
            produtos.add(produto);
            return produto;
        } else {
            return null;
        }
    }

    public static Produto findById(Long id) {
        return produtos.stream()
            .filter(p -> p.getId().equals(id))
            .findFirst()
            .orElse(null);
    }
}
```

```
public static List<Produto> findByCategoria(Categoria categoria) {  
    return produtos.stream()  
        .filter(p -> p.getCategoria().equals(categoria))  
        .toList();  
}  
  
public static List<Produto> findAll() {  
    return produtos;  
}  
}
```

Código Fonte 10 - Versão final da classe de repositório de produtos
Fonte: Autor (2023)

Precisamos agora de classes que interajam com os clientes da nossa aplicação. Para esse conjunto de classes, criaremos um pacote para organizá-las. Será o pacote *view*. Nele, teremos um conjunto de classes que receberão solicitações dos usuários do programa e, com o apoio das demais classes existentes, fornecerão o que for solicitado.

Vamos lá.

1.3 O pacote View

Dentro do pacote view criaremos classes que terão a responsabilidade de interação com o usuário do programa. É o que o usuário poderá operar. São as telas do nosso programa.

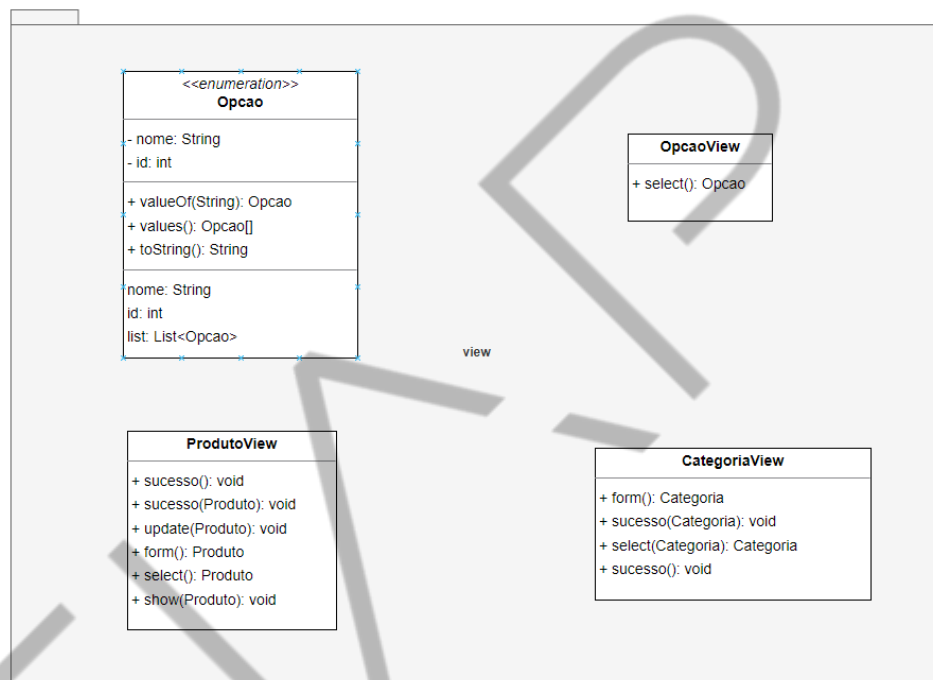


Figura 3 - pacote view
Fonte: Autor (2023)

1.3.1 A ENUMERAÇÃO DE OPÇÕES DO MENU

Criaremos inicialmente uma enumeração para conter o conjunto das opções disponíveis para o nosso programa. Funcionará como itens de menu.

```
package br.com.fiap.nano.produtos.view;
import java.util.Arrays;
import java.util.List;

public enum Opcao {

    CADASTRAR_CATEGORIA(1, "Cadastrar Categoria"),
```

```
CADASTRAR_PRODUTO(2, "Cadastrar Produto"),
ALTERAR_PRODUTO(3, "Alterar dados do Produto"),
CONSULTAR_PRODUTO_POR_ID(4, "Consultar Produto Por ID"),
CONSULTAR_PRODUTO_POR_CATEGORIA(5, "Consultar Produto Por
Categoria"),
ENCERRAR_SISTEMA(6, "Encerrar Sistema");

private String nome;
private int id;

Opcao(int id, String nome) {
    this.id = id;
    this.nome = nome;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public static List<Opcao> getList() {
    return Arrays.asList(Opcao.values());
}

@Override
public String toString() {
    return nome;
}
}
```

Código Fonte 11 - A enumeração de Opções do Sistema
Fonte: Autor (2023)

Perfeito! Temos a enumeração que nos auxiliará a exibir itens para o usuário do programa escolher o que deseja fazer. Como essas opções não mudarão durante a execução do sistema, uma enumeração é a melhor forma de representar itens de menu.

A nossa intenção, neste curso, não é criar interfaces gráficas bem elaboradas, estamos trabalhando em uma linguagem com foco no *backend*. Existem linguagem e tecnologias que cumprem muito bem esse papel de interfaces gráficas com o usuário.

Para mostrar ao usuário as opções disponíveis nós vamos utilizar uma caixa de seleção que pode ser criada com o `JOptionPane` por meio do método `ShowInputDialog`. Então vamos lá.

1.3.2 A CLASSE OPÇÃO VIEW

Chegou a hora de fazer a classe que de fato será a lista de opções do nosso menu. A classe `OpcaoView` terá o método `select` que será responsável por apresentar na tela uma caixa de seleção da qual o usuário poderá escolher para interagir com o programa.

```
package br.com.fiap.nano.produtos.view;

import javax.swing.*;
import java.util.Arrays;
import java.util.List;

public class OpcaoView {

    public static Opcao select() {

        Opcao ret = (Opcao) JOptionPane.showInputDialog(
            null, // componente pai. Como não temos será null
            "Selecione uma opção",
            "Menu",
            JOptionPane.QUESTION_MESSAGE,
            null, // icone
            Opcao.values(), // Número da opção
            Opcao.CADASTRAR_PRODUTO);

        return ret != null ? ret : Opcao.ENCERRAR_SISTEMA;
    }
}
```

Código Fonte 12 - A classe Opcao View
Fonte: Autor (2023)

Percebam que estou usando a enumeração de opções e chamando um método de nome `values`. Esse método traz todos os itens da enumeração. Entretanto, percebam também que uma opção deverá ser a opção padrão, que é a opção que

aparece selecionada na caixa de seleção. Para isso, nós vamos deixar a opção de cadastramento de produto.

Então veja, O método *ShowInputDialog* retorna *Object*, nós fizemos um *cast* para *Opção*, pois estamos utilizando a enumeração de *Opções*. O resultado da opção selecionada nós colocamos na variável de nome *ret*. Ao final, fazemos um *if* ternário para verificar se *ret* está diferente de *null*. Sendo diferente, nós retornaremos o valor selecionado, caso contrário a opção será encerrar o sistema.

1.3.3 A CLASSE CATEGORIA VIEW

Agora que já estamos familiarizados com o funcionamento do *JOptionPane.showInputDialog*, nós podemos avançar mais um pouco e criar a *view* de categorias. Ótimo, entretanto, existe uma pequena diferença. As opções do menu vinham de uma enumeração. Já as categorias vêm de um repositório que a nossa *view* não sabe qual é, mas sabe que receberá uma lista de categorias e isso é suficiente. Excelente! Pois não ficamos dependentes, podemos trocar a origem dos dados sem impacto nessa camada da aplicação.

Vamos codificar nossa *view* de categorias:

```
import br.com.fiap.nano.produtos.model.Categoria;
import br.com.fiap.nano.produtos.repository.CategoriaCollectionRepository;
import javax.swing.*;

public class CategoriaView {

    static CategoriaCollectionRepository repository;

    public static Categoria select(Categoria categoria) {

        Categoria ret = (Categoria) JOptionPane.showInputDialog(
            null, // componente pai. Como não temos será null
            "Selecione uma categoria",
            "Menu",
            JOptionPane.QUESTION_MESSAGE,
            null, // icone
            repository.findAll().toArray(), // Número da opção
            categoria == null ? 1 : categoria);

        return ret;
    }
}
```

```
}
```

Código Fonte 13 - A classe Categoria View
Fonte: Autor (2023)

Muito simples não é mesmo? Nós criamos uma variável *repository* do tipo *CategoriaCollectionRepository*. Esse é o objeto que nos proverá os dados de categoria. Excelente! E nós fizemos a chamada para que *repository* execute o método *findAll* e, em seguida, transforme a listagem em *array*. Precisamos dessa caixa de seleção em três situações:

- 1ª) Quando estamos cadastrando um produto;
- 2ª) Quando estivermos consultando um produto por categoria;
- 3ª) Quando estamos cadastrando uma categoria.

Sendo assim, para atender a 3ª situação, devemos fornecer uma maneira (uma caixa de texto) para o usuário cadastrar a nova categoria. Vamos incluir um novo método à classe *CategoriaView*. O nome do método será *form*. Vamos aos códigos.

```
public static Categoria form() {  
    String nome = JOptionPane.showInputDialog("Informe o nome da Categoria");  
    return new Categoria(nome);  
}
```

Código Fonte 14 - Incluindo o método *form* à classe *Categoria View*
Fonte: Autor (2023)

A responsabilidade do método *form* é apresentar uma caixa de texto para o usuário e capturar o nome da categoria. Em seguida, criar uma instancia de categoria e, por fim, retorná-la para quem solicitou os serviços do método.

Agora, seria interessante criar uma mensagem de sucesso para que o usuário saiba que a categoria foi salva. Vamos fazer dois. Criaremos uma sobrecarga do método *sucesso*. Um com o Objeto *Categoria* que fora salvo. Nesse, a mensagem será mais detalhada, incluirá o nome da categoria que foi salva com sucesso. Entretanto, farei um mais genérico também.

```
public void sucesso() {
    JOptionPane.showMessageDialog(null, "Categoria salva com sucesso!");
}
public void sucesso(Categoria categoria) {
    System.out.println(categoria);
    JOptionPane.showMessageDialog(null, "Categoria " +
    categoria.getNome().toUpperCase() + " salva com sucesso!");
}
```

Código Fonte 15 - Métodos para mensagem de sucesso
Fonte: Autor (2023)

Excelente! Nossa view de Categorias está pronta. Veja abaixo a versão final.

```
package br.com.fiap.nano.produtos.view;
import br.com.fiap.nano.produtos.model.Categoria;
import br.com.fiap.nano.produtos.repository.CategoriaCollectionRepository;
import javax.swing.*;

public class CategoriaView {
    static CategoriaCollectionRepository repository;

    public static Categoria select(Categoria categoria) {

        Categoria ret = (Categoria) JOptionPane.showInputDialog(
            null, // componente pai. Como não temos será null
            "Selecione uma categoria",
            "Menu",
            JOptionPane.QUESTION_MESSAGE,
            null, // icone
            repository.findAll().toArray(), // Número da opção
            categoria == null ? 1 : categoria);
        return ret;
    }

    public static Categoria form() {
        String nome = JOptionPane.showInputDialog("Informe o nome da Categoria");
        return new Categoria(nome);
    }

    public void sucesso() {
        JOptionPane.showMessageDialog(null, "Categoria salva com sucesso!");
    }
    public void sucesso(Categoria categoria) {
        System.out.println(categoria);
        JOptionPane.showMessageDialog(null, "Categoria " +
        categoria.getNome().toUpperCase() + " salva com sucesso!");
    }
}
```

Código Fonte 16 - A versão final da classe Categoria View
Fonte: Autor (2023)

1.3.4 A CLASSE PRODUTO VIEW

Chegou a hora de criar a view de produtos. Seguiremos a mesma estratégia que estamos adotando com as outras classes view.

Vamos iniciar com o código que exibe os produtos cadastrados:

```
import br.com.fiap.nano.produtos.model.Categoria;
import br.com.fiap.nano.produtos.model.Produto;
import br.com.fiap.nano.produtos.repository.ProdutoCollectionRepository;

import javax.swing.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

public class ProdutoView {

    public static Produto select() {

        Produto ret = (Produto) JOptionPane.showInputDialog(
            null, // componente pai. Como não temos será null
            "Selecione uma produto",
            "Menu",
            JOptionPane.QUESTION_MESSAGE,
            null, // ícone
            ProdutoCollectionRepository.findAll().toArray(), // Número da opção
            1);
        return ret;
    }
}
```

Código Fonte 17 – Desenvolvendo a classe ProdutoView
Fonte: Autor (2023)

Também teremos o método form para capturar os dados do produto que o usuário digitar. Vamos implementá-lo.

```
public static Produto form() {  
    Categoria categoria = null;  
  
    do {  
        categoria = CategoriaView.select(null);  
    } while (categoria == null);  
  
    String nome = "";  
  
    do {  
        nome = JOptionPane.showInputDialog("Nome do Produto");  
  
        if (nome.length() < 3)  
            JOptionPane.showMessageDialog(null, "O nome do produto precisa ter no  
mínimo 3 dígitos");  
  
    } while (nome.equals(""));  
  
    String descricao = "";  
  
    do {  
        descricao = JOptionPane.showInputDialog("Descrição do Produto");  
  
        if (descricao.length() < 5)  
            JOptionPane.showMessageDialog(null, "A descrição do produto precisa  
ter no mínimo 5 dígitos");  
  
    } while (descricao.equals(""));  
  
    double p = 0;  
  
    do {  
        try {  
            p = Double.parseDouble(JOptionPane.showInputDialog("Preço do  
Produto"));  
        } catch (NumberFormatException e) {  
            p = 0;  
        }  
  
        if (p <= 0) JOptionPane.showMessageDialog(null, "Valor inválido");  
  
    } while (p <= 0);  
  
    BigDecimal preco = BigDecimal.valueOf(p);  
  
    Produto ret = new Produto();  
    ret.setNome(nome)  
        .setCategoria(categoria)  
        .setDescricao(descricao)  
        .setPreco(preco)  
        .setDataCadastro(LocalDateTime.now());  
}
```

```
    return ret;  
}
```

Código Fonte 18 - O método form para capturar os dados do produto
Fonte: Autor (2023)

Perceba que fizemos algumas validações no método form, pois podemos receber dados inválidos.

Repare também que utilizamos a classe BigDecimal para armazenar o preço do produto. É o ideal quando trabalhamos com valores.

Precisamos agora fazer um método para trabalhar a atualização de dados do produto.

```
public static void update(Produto produto) {  
    Categoria categoria = null;  
  
    do {  
        categoria = CategoriaView.select(produto.getCategoria());  
        produto.setCategoria(categoria);  
    } while (categoria == null);  
  
    String nome = "";  
  
    do {  
        nome = JOptionPane.showInputDialog("Nome do Produto",  
produto.getNome());  
  
        if (nome.length() < 3)  
            JOptionPane.showMessageDialog(null, "O nome do produto precisa ter no  
mínimo 3 dígitos");  
  
        produto.setNome(nome);  
    } while (nome.equals(""));  
  
    String descricao = "";  
  
    do {  
        descricao = JOptionPane.showInputDialog("Descrição do Produto",  
produto.getDescricao());  
  
        if (descricao.length() < 5)  
            JOptionPane.showMessageDialog(null, "A descrição do produto precisa  
ter no mínimo 5 dígitos");  
  
        produto.setDescricao(descricao);  
    } while (descricao.equals(""));  
}
```

Projeto

```
    } while (descricao.equals(""));

    double p = 0;

    do {

        try {
            p = Double.parseDouble(JOptionPane.showInputDialog("Preço do
Produto", produto.getPreco()));
        } catch (NumberFormatException e) {
            p = 0;
        }

        if (p <= 0) JOptionPane.showMessageDialog(null, "Valor inválido");

    } while (p <= 0);

    BigDecimal preco = BigDecimal.valueOf(p);
    produto.setPreco(preco);

    sucesso(produto);
    show(produto);
}
```

Código Fonte 19 - Método para a atualização de dados do produto
Fonte: Autor (2023)

Código muito semelhante ao cadastro, a diferença é que os campos estão preenchidos e o usuário altera o que for necessário.

Por fim, vamos agora criar os métodos para mensagem de sucesso. E também o método para exibir os dados de um produto salvo.

```
public static void sucesso() {
    JOptionPane.showMessageDialog(null, "Produto salvo com sucesso!");
}

public static void sucesso(Produto produto) {
    System.out.println(produto);
    JOptionPane.showMessageDialog(null, "Produto " +
produto.getNome().toUpperCase() + " salvo com sucesso!");
}

public static void show(Produto p) {
    System.out.println(p);
    String produtoString = String.format("PRODUTO: " + p.getNome() +
System.lineSeparator() + "DESCRIÇÃO: " + p.getDescricao() +
```

Projeto

```
System.lineSeparator() + "CATEGORIA: " + p.getCategoria().toString() +  
System.lineSeparator() + "PREÇO: %,.2f", p.getPreco());  
JOptionPane.showMessageDialog(null, produtoString);  
}  
}
```

Código Fonte 20 - Métodos para mensagem de sucesso
Fonte: Autor (2023)

Concluimos a classe ProdutoView. Veja logo abaixo o código por completo:

```
package br.com.fiap.nano.produtos.view;  
  
import br.com.fiap.nano.produtos.model.Categoria;  
import br.com.fiap.nano.produtos.model.Produto;  
import br.com.fiap.nano.produtos.repository.ProdutoCollectionRepository;  
  
import javax.swing.*;  
import java.math.BigDecimal;  
import java.time.LocalDateTime;  
  
public class ProdutoView {  
  
    public static Produto select() {  
        // @formatter:off  
        Produto ret = (Produto) JOptionPane.showInputDialog(  
            null, // componente pai. Como não temos será null  
            "Selecione uma produto",  
            "Menu",  
            JOptionPane.QUESTION_MESSAGE,  
            null, // icone  
            ProdutoCollectionRepository.findAll().toArray(), // Número da opção  
            1);  
  
        return ret;  
        // @formatter:on  
    }  
  
    public static Produto form() {  
  
        Categoria categoria = null;  
  
        do {  
            categoria = CategoriaView.select(null);  
        } while (categoria == null);  
  
        String nome = "";
```

Projeto

```
do {
    nome = JOptionPane.showInputDialog("Nome do Produto");

    if (nome.length() < 3)
        JOptionPane.showMessageDialog(null, "O nome do produto precisa ter no
mínimo 3 dígitos");

    } while (nome.equals(""));

    String descricao = "";

    do {
        descricao = JOptionPane.showInputDialog("Descrição do Produto");

        if (descricao.length() < 5)
            JOptionPane.showMessageDialog(null, "A descrição do produto precisa
ter no mínimo 5 dígitos");

        } while (descricao.equals(""));

    double p = 0;

    do {

        try {
            p = Double.parseDouble(JOptionPane.showInputDialog("Preço do
Produto"));
        } catch (NumberFormatException e) {
            p = 0;
        }

        if (p <= 0) JOptionPane.showMessageDialog(null, "Valor inválido");

    } while (p <= 0);

    BigDecimal preco = BigDecimal.valueOf(p);

    Produto ret = new Produto();
    ret.setNome(nome)
        .setCategoria(categoria)
        .setDescricao(descricao)
        .setPreco(preco)
        .setDataCadastro(LocalDateTime.now());

    return ret;
}

public static void update(Produto produto) {

    Categoria categoria = null;

    do {
```

Projeto

```
        categoria = CategoriaView.select(produto.getCategoria());
        produto.setCategoria(categoria);
    } while (categoria == null);

    String nome = "";

    do {
        nome = JOptionPane.showInputDialog("Nome do Produto",
        produto.getNome());

        if (nome.length() < 3)
            JOptionPane.showMessageDialog(null, "O nome do produto precisa ter no
mínimo 3 dígitos");

        produto.setNome(nome);

    } while (nome.equals(""));

    String descricao = "";

    do {
        descricao = JOptionPane.showInputDialog("Descrição do Produto",
        produto.getDescricao());

        if (descricao.length() < 5)
            JOptionPane.showMessageDialog(null, "A descrição do produto precisa
ter no mínimo 5 dígitos");

        produto.setDescricao(descricao);
    } while (descricao.equals(""));

    double p = 0;

    do {

        try {
            p = Double.parseDouble(JOptionPane.showInputDialog("Preço do
Produto", produto.getPreco()));
        } catch (NumberFormatException e) {
            p = 0;
        }

        if (p <= 0) JOptionPane.showMessageDialog(null, "Valor inválido");

    } while (p <= 0);

    BigDecimal preco = BigDecimal.valueOf(p);
    produto.setPreco(preco);

    sucesso(produto);
    show(produto);
}

public static void sucesso() {
```

```
        JOptionPane.showMessageDialog(null, "Produto salvo com sucesso!");
    }

    public static void sucesso(Produto produto) {
        System.out.println(produto);
        JOptionPane.showMessageDialog(null, "Produto " +
        produto.getNome().toUpperCase() + " salvo com sucesso!");
    }

    public static void show(Produto p) {
        System.out.println(p);
        String produtoString = String.format("PRODUTO: " + p.getNome() +
        System.lineSeparator() + "DESCRIÇÃO: " + p.getDescricao() +
        System.lineSeparator() + "CATEGORIA: " + p.getCategoria().toString() +
        System.lineSeparator() + "PREÇO: %,2f", p.getPreco());
        JOptionPane.showMessageDialog(null, produtoString);
    }
}
```

Código Fonte 21 - Código completo da classe ProdutoView
Fonte: Autor (2023)

Falta muito pouco! Agora precisamos codificar o método main da classe App. A classe App tem apenas a responsabilidade de mostrar as opções ao usuário e, quando o usuário escolher uma determinada opção, ele será direcionado para o método que chamará a view correspondente.

1.4O pacote Padrão

Uma boa prática é colocar a classe que inicia o nosso programa no pacote padrão. O pacote padrão é geralmente a raiz da pasta SRC ou no pacote padrão (aquele que termina com o nome da aplicação). No nosso projeto, nós criamos um pacote `br.com.fiap.nano.produtos`.

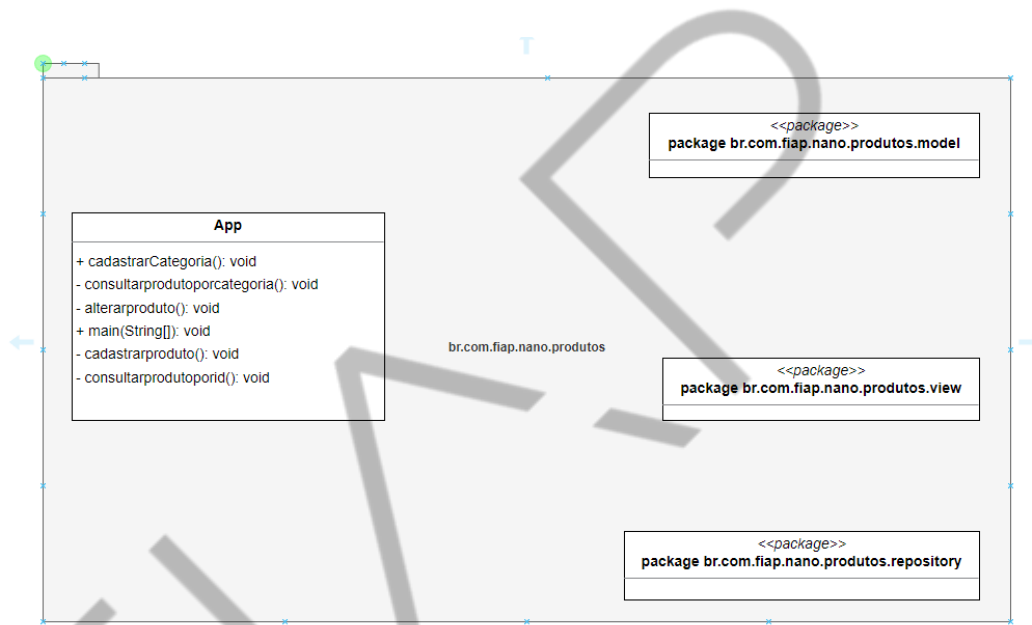


Figura 4 - O pacote padrão
Fonte: Autor (2023)

É neste pacote que criaremos a classe `App`.

1.4.1 A CLASSE APP

A classe `App` será o ponto de partida do nosso programa. Iniciaremos o desenvolvimento implementando o método `main`.

```
package br.com.fiap.nano.produtos;

import br.com.fiap.nano.produtos.view.Opcao;
import br.com.fiap.nano.produtos.view.OpcaoView;

import javax.swing.*;
```

Projeto

```
import java.util.List;

public class App {

    public static void main(String[] args) {

        Opcao opcao = null;

        do {
            opcao = OpcaoView.select();
            switch (opcao) {
                case CADASTRAR_CATEGORIA -> cadastrarCategoria();
                case CADASTRAR_PRODUTO -> cadastrarproduto();
                case ALTERAR_PRODUTO -> alterarproduto();
                case CONSULTAR_PRODUTO_POR_ID -> consultarprodutoporid();
                case CONSULTAR_PRODUTO_POR_CATEGORIA ->
                    consultarprodutoporcategoria();
            }
        } while (opcao != Opcao.ENCERRAR_SISTEMA);
    }
}
```

Código Fonte 22 - Iniciando o desenvolvimento da classe App
Fonte: Autor (2023)

Iniciamos o programa mostrando para o usuário as opções disponíveis. Utilizamos para isso um laço de repetição que deverá ser repetido até que a opção escolhida seja de encerrar o sistema. Dentro desse laço de repetição do tipo *do while* nós temos um switch para verificar qual foi a opção escolhida pelo usuário. Para cada opção criaremos um método. Então seguiremos adicionando os métodos na classe App pela ordem que consta no switch.

O passo seguinte é criar o método para quando o usuário escolher a funcionalidade de Cadastrar Categoria:

```
public static void cadastrarCategoria() {
    CategoriaView view = new CategoriaView();
    Categoria categoria = view.form();
    CategoriaCollectionRepository.save(categoria);
    view.sucesso(categoria);
}
```

Código Fonte 23 - Cadastramento de Categoria
Fonte: Autor (2023)

Projeto

Agora vamos criar o método para quando o usuário escolher a funcionalidade de Cadastrar Produto:

```
private static void cadastrarproduto() {  
    Produto produto = ProdutoView.form();  
    ProdutoCollectionRepository.save(produto);  
    ProdutoView.sucesso(produto);  
}
```

Código Fonte 24 - Cadastramento de Produto
Fonte: Autor (2023)

O método para alteração do produto será exibido abaixo:

```
private static void alterarproduto() {  
    Produto produto = ProdutoView.select();  
    ProdutoView.update(produto);  
}
```

Código Fonte 25 - Alterar Produto
Fonte: Autor (2023)

Também faremos o método para consultar produto por ID

```
private static void consultarprodutoporid() {  
    Long id = 0L;  
    do {  
        try {  
            id = Long.parseLong(JOptionPane.showInputDialog("Informe o id do  
produto"));  
        } catch (Exception e) {  
            JOptionPane.showMessageDialog(null, "Id inválido!");  
        }  
    } while (id <= 0);  
  
    Produto p = ProdutoCollectionRepository.findById(id);  
    if (p != null) {  
        ProdutoView.show(p);  
    } else {  
        JOptionPane.showMessageDialog(null, "Produto não encontrado!");  
    }  
}
```

Código Fonte 26 - Consultar produto por Id
Fonte: Autor (2023)

Projeto

Também faremos método para consultar produto por Categoria.

```
private static void consultarprodutoporcategoria() {
    Categoria categoria = CategoriaView.select(null);
    List<Produto> produtos =
    ProdutoCollectionRepository.findByCategoria(categoria);
    if (produtos.size() == 0)
        JOptionPane.showMessageDialog(null, "Não encontramos produtos
    cadastrados para a categoria " + categoria.getNome());
    produtos.forEach(System.out::println);
    produtos.forEach(ProdutoView::show);
}
```

Código Fonte 27 - Consultar produtos por categoria
Fonte: Autor (2023)

No final a nossa classe APP ficará assim:

```
package br.com.fiap.nano.produtos;

import br.com.fiap.nano.produtos.model.Categoria;
import br.com.fiap.nano.produtos.model.Produto;
import br.com.fiap.nano.produtos.repository.CategoriaCollectionRepository;
import br.com.fiap.nano.produtos.repository.ProdutoCollectionRepository;
import br.com.fiap.nano.produtos.view.CategoriaView;
import br.com.fiap.nano.produtos.view.Opcao;
import br.com.fiap.nano.produtos.view.OpcaoView;
import br.com.fiap.nano.produtos.view.ProdutoView;

import javax.swing.*;
import java.util.List;

public class App {
    public static void main(String[] args) {

        List<Categoria> categorias = CategoriaCollectionRepository.findAll();

        Opcao opcao = null;

        do {
            opcao = OpcaoView.select();
            switch (opcao) {
                case CADASTRAR_CATEGORIA -> cadastrarCategoria();
                case CADASTRAR_PRODUTO -> cadastrarproduto();
                case ALTERAR_PRODUTO -> alterarproduto();
                case CONSULTAR_PRODUTO_POR_ID -> consultarprodutoporid();
                case CONSULTAR_PRODUTO_POR_CATEGORIA ->
                consultarprodutoporcategoria();
            }
        } while (opcao != Opcao.ENCERRAR_SISTEMA);
    }
}
```

```
private static void consultarprodutoporcategoria() {
    Categoria categoria = CategoriaView.select(null);
    List<Produto> produtos =
    ProdutoCollectionRepository.findByCategoria(categoria);
    if (produtos.size() == 0)
        JOptionPane.showMessageDialog(null, "Não encontramos produtos
    cadastrados para a categoria " + categoria.getNome());
    produtos.forEach(System.out::println);
    produtos.forEach(ProdutoView::show);
}

private static void consultarprodutoporid() {
    Long id = 0L;
    do {
        try {
            id = Long.parseLong(JOptionPane.showInputDialog("Informe o id do
    produto"));
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null, "Id inválido!");
        }
    } while (id <= 0);

    Produto p = ProdutoCollectionRepository.findById(id);
    if (p != null) {
        ProdutoView.show(p);
    } else {
        JOptionPane.showMessageDialog(null, "Produto não encontrado!");
    }
}

private static void alterarproduto() {
    Produto produto = ProdutoView.select();
    ProdutoView.update(produto);
}

private static void cadastrarproduto() {
    Produto produto = ProdutoView.form();
    ProdutoCollectionRepository.save(produto);
    ProdutoView.sucesso(produto);
}

public static void cadastrarCategoria() {
    CategoriaView view = new CategoriaView();
    Categoria categoria = view.form();
    CategoriaCollectionRepository.save(categoria);
    view.sucesso(categoria);
}
}
```

Projeto

Agora podemos testar a nossa aplicação e ver se está tudo funcionando perfeitamente. Caso apareça algum erro a gente vai fazer a devida correção.

EMSE

1.5 O programa em funcionamento

Chegou o momento mais esperado. O programa em funcionamento. Faremos uma breve demonstração nos itens seguintes.

1.5.1 MENU DE OPÇÕES

A primeira tela que será exibida ao carregar o programa será o menu de opções.

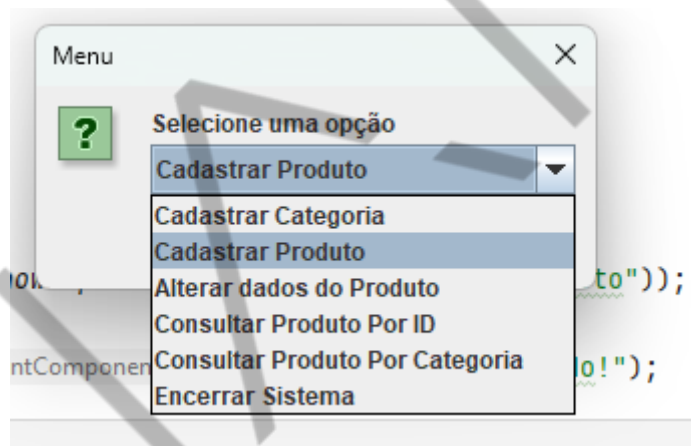


Figura 5 - Menu de opções
Fonte: Autor (2023)

1.5.2 CADASTRANDO UMA NOVA CATEGORIA

Estamos cadastrando uma nova categoria. Digitamos Livros. Perceba que já existe uma categoria com esse nome o programa não poderá permitir duplicidades.

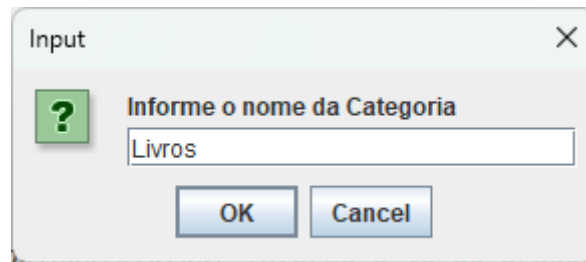


Figura 6 - Cadastrando uma categoria
Fonte: Autor (2023)

Em caso de sucesso, a seguinte tela deverá ser exibida para o usuário:

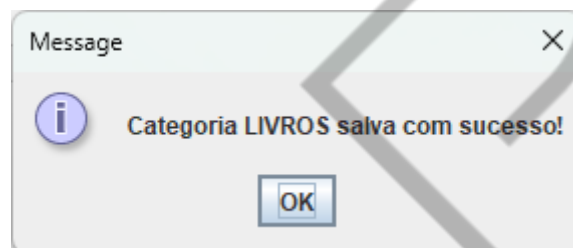


Figura 7 - Sucesso no cadastramento de categoria
Fonte: Autor (2023)

1.5.3 CADASTRANDO UM NOVO PRODUTO

Ao selecionar cadastramento de produto, primeiramente aparecerá a tela para escolher a categoria.

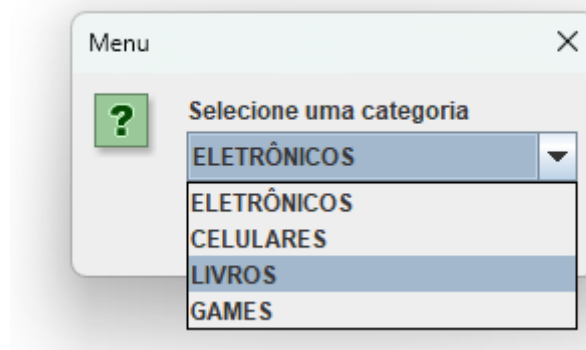


Figura 8 - Categorias existentes
Fonte: Autor (2023)

Projeto

No momento escolhi livros em seguida é exibida a caixa de texto para darmos o nome do livro.

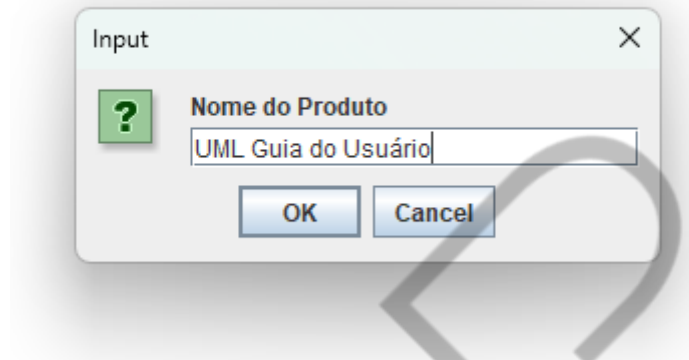


Figura 9 - Dando nome ao produto
Fonte: Autor (2023)

Digitamos UML Guia do Usuário como nome do nosso produto da categoria Livros. Em seguida será solicitada a descrição do produto.

Digitei no campo descrição:

“O mais avançado tutorial sobre Unified Modeling Language (UML) ”.

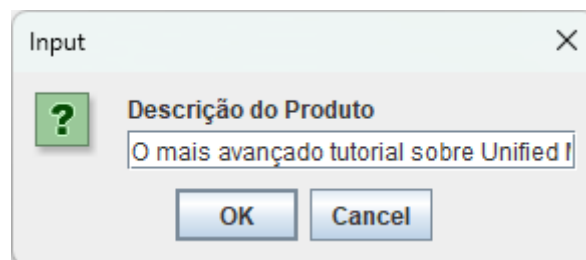


Figura 10 - Inserindo a descrição do produto
Fonte: Autor (2023)

Em seguida aparece a tela para eu inserir o preço do produto.

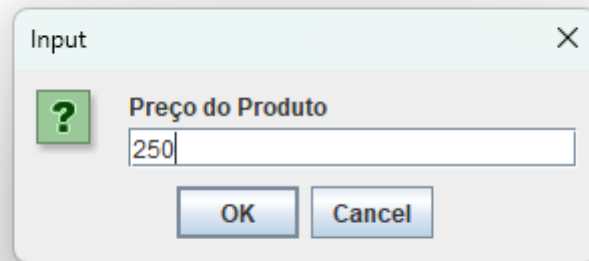


Figura 11 - Inserindo o preço do produto
Fonte: Autor (2023)

Após digitar o preço do produto, exibiremos a mensagem de sucesso!

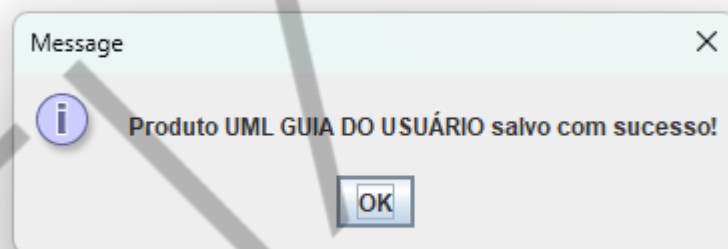


Figura 12 - Mensagem de sucesso ao cadastrar um produto.
Fonte: Autor (2023)

1.5.4 ALTERANDO DADOS DO PRODUTO

Ao selecionar a opção alterar dados do produto, a seguinte tela será exibida:

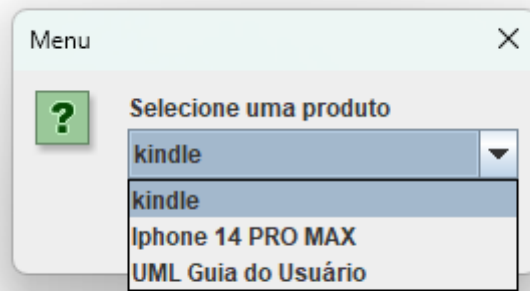


Figura 13 - Alterando dados dos Produtos
Fonte: Autor (2023)

Escolhemos o livro, UML Guia do Usuário. Os campos aparecem para que alteremos os dados.

Primeira tela é a de categoria. Manteremos como Livros.

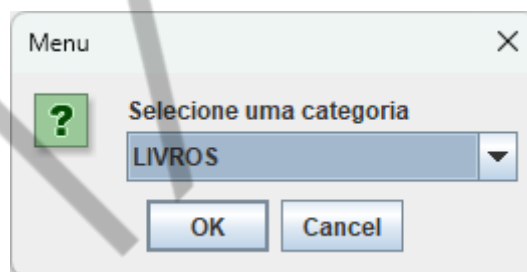


Figura 14 - Tela para alteração da categoria do produto
Fonte: Autor (2023)

Ao escolher livros e clicar em OK, a tela para nome do produto será exibida.

Vou alterar para mostrar que o livro está na segunda edição digitando: UML Guia do Usuário - 2ª Edição.

Projeto

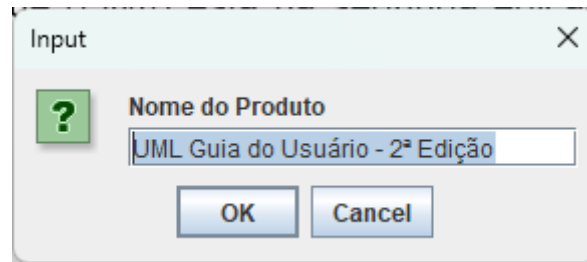


Figura 15 - Alterando o nome do produto
Fonte: Autor (2023)

Para os demais dados, mantive os valores existentes. Vejamos a mensagem de sucesso!

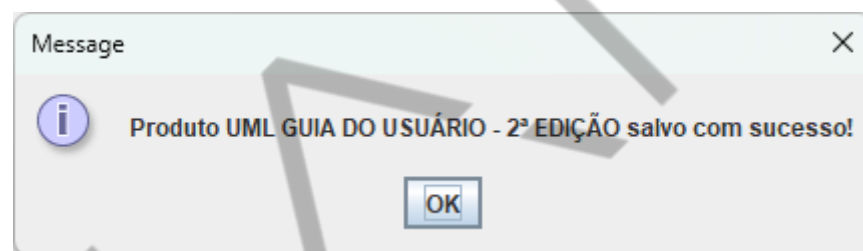


Figura 16 - Mensagem de Sucesso
Fonte: Autor (2023)

Em seguida, mostramos todos os dados do produto. Vejamos:

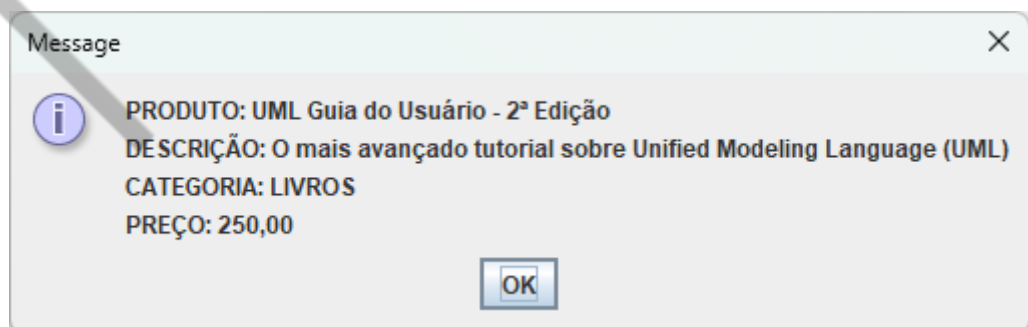


Figura 17 - Todos os dados do produto
Fonte: Autor (2023)

1.5.5 CONSULTANDO PRODUTO POR ID

Outra funcionalidade disponível para o usuário é a consulta de produto por Id. Veja como ficou a tela:

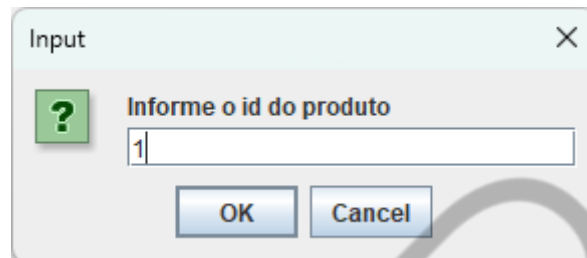


Figura 18 - Consultando produto por Id
Fonte: Autor (2023)

Digitei o número 1 para carregar os dados do produto com Id igual a 1.

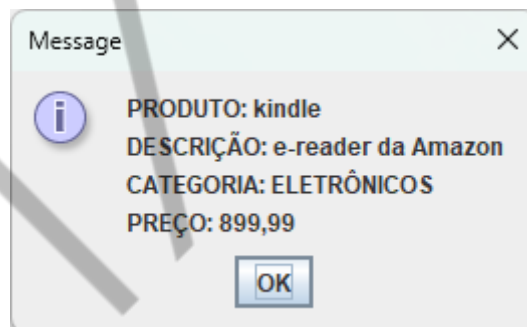


Figura 19 - Dados do Produto com Id iguala 1
Fonte: Autor (2023)

1.5.6 LISTANDO PRODUTOS POR CATEGORIA

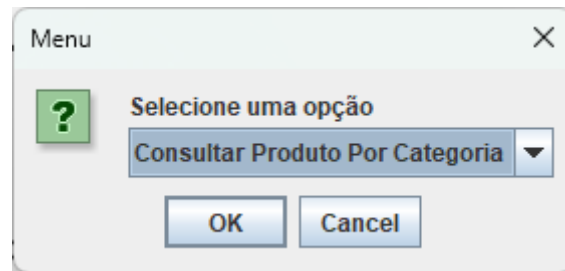


Figura 20 - Selecionar produto por categoria
Fonte: Autor (2023)

Ao selecionar a opção de consulta de produtos por categoria, a tela para selecionar a categoria desejada será exibida. Vejamos:

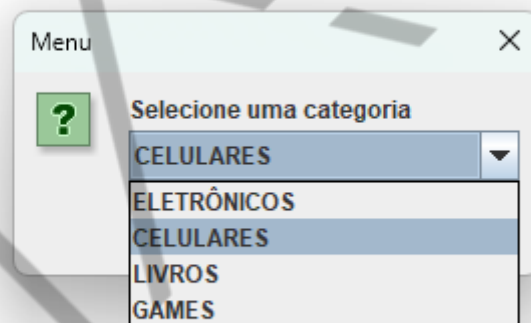


Figura 21 - Selecionando a categoria para listagem
Fonte: Autor (2023)

Selecionei agora a categoria celulares. Veja o resultado:



Figura 22 - Apresentando dados do produto de categoria Celulares
Fonte: Autor (2023)

Por fim, podemos encerrar o sistema.

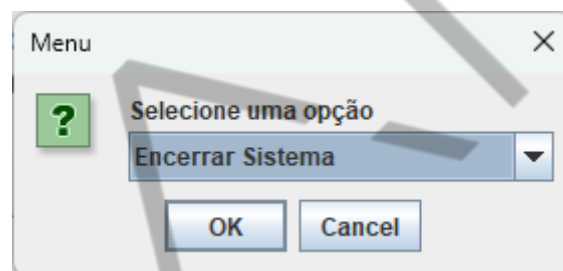


Figura 23 - Encerrando o sistema
Fonte: Autor (2023)

Concluimos assim este capítulo. Codificamos bastante, praticando os conceitos aprendidos durante o treinamento. Temos certeza de que você está apto para desenvolver soluções Java com respeito à múltiplas camadas, fazendo uso de boas práticas de desenvolvimento.

Nunca pare de estudar!

REFERÊNCIAS

BARNES, David J. **Programação Orientada a Objetos com Java: Uma introdução Prática Utilizando Blue J**. São Paulo: Pearson, 2004.

COLHO, Alex. **Java com Orientação a Objetos**. Rio de Janeiro: Ciência Moderna, 2012.

DEITEL P, DEITEL H. **Java Como Programar, 10ª ed.** Pearson, 2016.

HORSTMANN, Cay; CORNELL, Gary. **Core Java. Volume I - Fundamentos. 8.ed.** São Paulo: Pearson 2009.

SIERRA, Kathy; BATES, Bert. **Use a cabeça! Java**. Rio de Janeiro: Alta Books, 2010